

Lebanese American University
Department of Computer Science & Mathematics

VR Classroom Teacher Evaluation System

Capstone Final Report

Project By:
Ricardo Salloum (ID: 202303922)

Supervisor:
Dr. Danielle Azar

Course: CSC599 Capstone Project
Semester: Spring 2026
Date: April 26, 2026

SIGNATURE PAGE

Student Signature: _____

Supervisor Signature: _____

Abstract

This report presents the design, implementation, and evaluation of a VR Classroom Teacher Evaluation System - a Unity-based simulation in which virtual students attend a real lecture, pay attention according to individually modeled attention profiles, and subsequently take a multiple-choice exam. The core innovation is a two-state Markov chain attention model that produces psychologically realistic zone-out patterns: attention lapses affect coherent bursts of content rather than random isolated words, leading to tier-separated exam performance without artificial hard caps.

The system integrates four major technical components: live lecture transcription via OpenAI Whisper, per-student attention filtering and context management in Python, retrieval-augmented generation (RAG) for MCQ answering, and Kokoro TTS enabling students to ask questions aloud during the lecture. Experimental results confirm that the system faithfully models attention-driven retention: Tier A students (high/perfect attention) average 21.7/25, Tier B (medium) averages 17.0/25, and Tier C (low) averages 10.7/25. All agents score exactly 0/25 when the exam is unrelated to the lecture, validating strict context isolation. The system also demonstrates that pedagogical repetition helps lower-attention tiers most, consistent with theoretical predictions of the Markov model.

Table of Contents

1. Introduction	1
2. Literature Review / Related Work	3
3. System Analysis and Design	7
4. Implementation	11
5. Testing and Evaluation	16
6. Entrepreneurial / Innovation Aspects	18
7. Project Management and Teamwork	20
8. Results and Discussion	21
9. Conclusion and Future Work	23
References	25

1. Introduction

1.1 Background and Motivation

Classroom attention is one of the most consequential and least measurable variables in education. A student who physically attends a lecture may retain anywhere from nearly everything to almost nothing, depending on their cognitive engagement at any given moment. Traditional academic tools - attendance sheets, participation grades, end-of-term exams - capture only coarse proxies of this variable and provide teachers with feedback too late to act on.

Advances in artificial intelligence, particularly large language models (LLMs), neural text-to-speech, and automatic speech recognition (ASR), now make it feasible to simulate this process computationally. Virtual students can be programmed with attention profiles, can "listen" to real lectures, and can answer exam questions using only the content they "retained." The result is a closed-loop teacher evaluation environment: a professor delivers a lecture, and the system immediately produces tier-stratified exam statistics revealing which questions were hardest, which concepts were most poorly retained, and how repetition affects scores across attention levels.

1.2 Problem Statement

No existing tool allows a professor to instantly and objectively measure the impact of their lecture on students of varying attention levels. Simulated student systems either use trivially uniform attention (all students hear the same thing) or apply

random word-level masking that does not reflect the coherent burst structure of real cognitive zone-outs. This project addresses both gaps.

The central research question is: can a computationally modeled attention system, applied to real lecture transcripts and evaluated through LLM-driven MCQ answering, produce exam score distributions that are (a) tier-separated without hard caps, (b) sensitive to lecture quality and content coverage, and (c) sensitive to pedagogical repetition?

1.3 Research Gap and Project Objectives

Existing educational simulation work focuses on student modelling at the course level (e.g., knowledge tracing, mastery learning) rather than the lecture level. No published system combines real-time ASR transcription, probabilistic attention modelling, retrieval-augmented LLM answering, and neural TTS into a single VR-integrated pipeline. The objectives of this project are:

- Design and implement a two-state Markov attention model that produces coherent, burst-structured zone-outs with tier-separated steady-state retention.
- Build a full pipeline from live microphone input → Whisper transcription → per-student filtered context → RAG-based MCQ answering.
- Validate that agents are strictly context-grounded (zero score on unrelated content).
- Demonstrate repetition sensitivity consistent with Markov theory.
- Integrate the entire system into a VR Unity classroom for live demonstration.
-

1.4 Significance and Potential Impact

If validated, this system can serve as a lecturer preparation tool, allowing professors to test their lecture notes against a simulated class before delivering them live. It can also inform curriculum design by quantifying which concepts require repetition for lower-attention learners. At the research level, the Markov attention model provides a principled, analytically tractable framework for studying repetition effects in educational content.

2. Literature Review / Related Work

2.1 Attention Modelling in Education

Research in cognitive psychology consistently shows that student attention during lectures is not uniformly distributed. Studies by Risko et al. (2012) found that mind-wandering episodes during lectures follow bursty patterns. Students zone out for a span of seconds to minutes before re-engaging, rather than losing isolated words at random. This empirical observation directly motivates the Markov burst model used in this project.

Bunce et al. (2010) further characterized lecture attention as oscillatory, with characteristic drops in engagement occurring approximately every 10 to 15 minutes. Their work supports the design decision to model attention as a stochastic process rather than as a fixed tier-level scalar: even a high-attention student will periodically experience short lapses. The Markov model captures this by assigning a non-zero p_{dropout} to the high attention profile (0.035), allowing rare but occasional zone-out bursts even for the best-performing students.

The analytical tractability of the Markov chain (steady-state $P(\text{attentive}) = p_{\text{recover}} / (p_{\text{dropout}} + p_{\text{recover}})$) provides a closed-form prediction for expected retention, allowing parameter choices to be justified theoretically rather than tuned empirically.

2.2 Automatic Speech Recognition

OpenAI Whisper (Radford et al., 2022) is an encoder-decoder Transformer trained on 680,000 hours of multilingual audio, achieving near-human word error rates on English transcription. The base model (74M parameters) was selected for this project as a balance between transcription quality and inference speed on the GTX 1050. VoiceController.cs uploads 30-second audio windows every 30 seconds to Agent_Voice.py, which transcribes and appends the cleaned text to the running lecture transcript. In practice, on a standard CPU the base model transcribes 30 seconds of audio in approximately 4 to 8 seconds, keeping the pipeline well within real-time constraints. In addition to raw transcription, the system applies a post-processing step that strips common spoken filler words - such as "uh", "um", "like", and "you know" - using regex-based pattern matching. This cleanup prevents filler words from polluting the student context and degrading downstream RAG retrieval quality.

2.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. (2020), combines dense retrieval over a document store with a generative LLM to ground outputs in a specific corpus. Each student's Markov-filtered lecture context serves as a personal, isolated knowledge base. Sentence-BERT (all-MiniLM-L6-v2)

embeds context chunks into a FAISS IndexFlatL2 index; at query time the top-5 chunks are retrieved by embedding similarity and re-ranked by a cross-encoder (ms-marco-MiniLM-L-6-v2), which scores each (question, chunk) pair jointly for higher precision. Crucially, each student's FAISS index is built exclusively from their own filtered context. A student who zoned out cannot retrieve information from the missed segment, which is confirmed by the zero-score isolation experiment in Section 5.3.

2.4 Large Language Models for MCQ Answering

This project uses Qwen2.5-0.5B-Instruct (Alibaba Cloud, 2024), loaded locally via HuggingFace Transformers. The 0.5B model was chosen to run within the 4 GB VRAM budget of the GTX 1050; larger variants (1.5B, 7B) can be substituted on stronger hardware.

2.5 Text-to-Speech

Kokoro TTS (PyTorch version, 2024) is a lightweight, fully offline text-to-speech system supporting twelve voices with different English accents. Its low inference latency (under one second for typical 10–30 word questions on the GTX 1050) makes it suitable for real-time use during a live lecture. The returned WAV audio is decoded in Unity by a custom WavToAudioClip() parser and played immediately through an AudioSource component.

2.6 VR as a Teacher Evaluation Platform

Teacher evaluation systems traditionally rely on peer observation, student surveys, or post-hoc grade analysis - all of which are slow, subjective, or too coarse to isolate the effect of a single lecture. A simulation-based approach, where virtual

students with modelled attention profiles attend a lecture and respond to exams drawn from its content, enables objective and immediate feedback grounded in what was actually said. Unity is the dominant platform for real-time 3D and VR applications and was chosen for its robust scene management and community ecosystem. For this project, the VR Game Jam Template by ValemVR (<https://github.com/ValemVR/VR-Game-Jam-Template>) provided the foundation for VR locomotion and basic interaction, allowing development focus to remain on the teacher evaluation simulation rather than low-level VR mechanics.

2.7 Gap Analysis

No existing teacher evaluation tool simulates how attention-heterogeneous students process and retain lecture content in real time. No published system combines: (1) live ASR transcription of a real lecturer with filler-word cleanup, (2) per-student probabilistic attention filtering producing coherent, burst-structured knowledge gaps, (3) strictly context-grounded RAG-based MCQ answering that cannot draw on background knowledge, and (4) neural TTS enabling virtual students to ask questions mid-lecture - all integrated within a VR classroom built on a community template. This project addresses that gap, providing a complete, hardware-friendly, and open-source pipeline for real-time teacher evaluation through simulated student behaviour.

3. System Analysis and Design

3.1 Requirements Analysis

3.1.1 Functional Requirements

- FR1: The system shall transcribe live microphone audio to text in near-real-time using Whisper ASR.
- FR2: Each virtual student shall maintain a private, independently filtered lecture context.
- FR3: The attention model shall apply sentence-level blanking followed by word-level Markov masking.
- FR4: Students shall answer MCQ questions using only their filtered context via RAG + LLM.
- FR5: The QuestioningManager shall periodically select a student and generate a spoken question via TTS.
- FR6: Exam results shall be aggregated and reported by tier (mean, median, standard deviation).
- FR7: The teacher shall be able to configure attention types, timing parameters, and file paths via Unity Inspector.

3.1.2 Non-Functional Requirements

- NFR1 - Isolation: Agents must score 0/25 on exams whose content is entirely absent from the lecture.
- NFR2 - Tier Separation: Mean scores for tiers A, B, C must not overlap across test runs.
- NFR3 - Latency: A 25-question exam for 9 students must complete within 5 minutes on a standard GPU.
- NFR4 - Modularity: The three Python servers (Agent, Voice, TTS) must be independently restartable.
- NFR5 - Persistence: File paths and settings persist between Unity sessions via PlayerPrefs.

3.2 System Architecture

The system follows a client-server architecture where Unity (C#) acts as the orchestration client and three independent Python FastAPI servers provide AI services. All communication uses HTTP POST with JSON payloads.

Component	File(s)	Port	Responsibility
Unity Frontend	ClassroomManager.cs StudentAgent.cs	-	Scene orchestration, exam flow, UI
Voice Layer	VoiceController.cs Agent_Voice.py	5005	Microphone capture, Whisper transcription
TTS Layer	QuestioningManager.cs Agent_TTS.py	5003	Kokoro speech synthesis, WAV playback
Student AI	Agent.py BaseStudent.py	5002	Markov attention, RAG Q&A, MCQ answering
Settings / UI	PathSettingsMenu.cs SimpleSlideshow.cs	-	PlayerPrefs, PDF slide viewer
Analytics	ExamStats.cs	-	Mean, median, std dev per tier

3.2.1 Use Cases

The following primary use cases define the system's operational scenarios from the professor's perspective.

UC1 - Start Lecture: Professor clicks Start Lecture. VoiceController begins recording; QuestioningManager starts its cooldown. The system enters live lecture phase and begins accumulating Whisper transcripts.

UC2 - Student Asks a Question: At each cooldown tick a random student is selected. The current lecture snippet is sent to /generate_lecture_question; the returned question is synthesized by Kokoro and played through the AudioSource.

UC3 - Run Exam: Professor stops lecture and triggers the exam. The full transcript is sent to each student via `/send_lecture`; MCQ questions are submitted concurrently via `/answer_mcq`. Results are aggregated by tier and logged.

UC4 - Load Exam from File: Professor sets a local JSON path in `PathSettingsMenu`. `ClassroomManager` parses the file and makes MCQ questions available for the exam phase.

3.2.2 Data Flow

The system operates across three phases. During the Live Lecture Phase, audio is captured by Unity's Microphone API and buffered in a 1800-second `AudioClip`. Every 30 seconds, `VoiceController` extracts the most recent audio window, encodes it as a WAV byte array, and HTTP-POSTs it to `Agent_Voice.py` (`/transcribe`, port 5005). Whisper decodes the audio, applies filler-word cleanup, and returns a JSON response containing both the original and cleaned transcription. The cleaned text is appended to `ClassroomManager.loadedLectureText`, which accumulates the full lecture transcript incrementally across all snippet intervals.

During the Questioning Phase, `QuestioningManager` periodically selects a random student, generates a question via `/generate_lecture_question`, and sends the text to `Agent_TTS` (`/synthesize`). The returned WAV is decoded and played through Unity's `AudioSource`.

During the Exam Phase, `ClassroomManager` sends the full transcript to each student via `/send_lecture`, which applies the two-phase attention pipeline and

builds a per-student FAISS index. MCQ coroutines are then launched concurrently (subject to a semaphore), each posting to /answer_mcq to retrieve context, rerank, and prompt Qwen2.5-0.5B. The returned answer. The returned choice_index and confidence are recorded. After all students answer all questions, ExamStats.cs computes mean, median, and standard deviation per tier.

3.3 Attention Model Design

The attention model is the theoretical centrepiece of the project. Students exist in one of two states at each word: ATTENTIVE or ZONED_OUT. Transitions are governed by two parameters:

- p_{dropout} : probability per word of transitioning from ATTENTIVE to ZONED_OUT
- p_{recover} : probability per word of transitioning from ZONED_OUT back to ATTENTIVE

The steady-state probability of being attentive is analytically derived as:

$$P(\text{attentive}) = p_{\text{recover}} / (p_{\text{dropout}} + p_{\text{recover}})$$

Profile	Tier	p_{dropout}	p_{recover}	P(attentive) steady-state	Avg. Zone-out (words)
Perfect	A	0.000	1.000	100%	0
High	A	0.035	0.700	~92%	2.9
Medium	B	0.100	0.340	~69%	5.6
Low	C	0.190	0.220	~35%	12.5

3.4 Feasibility and Risk Analysis

The project is technically feasible on a standard GPU workstation (RTX 3060 or better). Key risks identified during design and their mitigations are:

- Risk: LLM inference latency exceeds acceptable exam duration.
Mitigation: Coroutine-based parallel MCQ submission with configurable concurrency limit (MAX_CONCURRENT_MCQ).
- Risk: Whisper produces incorrect transcriptions for domain-specific vocabulary. Mitigation: Filler-word cleanup pass in Agent_Voice.py; lecturer can review the loaded transcript before starting the exam.
- Risk: Students with no context (empty after masking) produce random answers. Mitigation: no_context flag returned by /answer_mcq so results can be flagged in analytics.
- Risk: Unity–Python API contract mismatches corrupt request payloads. Mitigation: all API inputs use typed Pydantic models on the Python side; Unity uses [Serializable] wrapper structs.

4. Implementation

4.1 Development Methodology

Each AI subsystem (ASR, attention model, RAG, TTS) was built and validated in isolation using unit tests before being integrated into the Unity scene. Integration was validated by end-to-end tests comparing agent scores against analytical predictions of the Markov model.

4.2 Tools and Technologies

Category	Technology	Version / Notes
VR Engine	Unity	Unity 6
Language (Unity)	C# / .NET	Unity Scripting API
Language (backend)	Python	3.11+
API Framework	FastAPI + Uvicorn	Async HTTP server
ASR	OpenAI Whisper	'base' model, CPU/GPU

Embeddings	Sentence-BERT	all-MiniLM-L6-v2
Vector Store	FAISS	IndexFlatL2
Re-ranker	Cross-Encoder	ms-marco-MiniLM-L-6-v2
LLM	Qwen2.5 0.5B	Quantized via HuggingFace
TTS	Kokoro	PyTorch

4.3 Module Descriptions

4.3.1 ClassroomManager.cs - Scene Orchestration

ClassroomManager is the Unity-side hub. It spawns StudentAgent GameObjects, manages the VoiceController and QuestioningManager references, collects Whisper transcript snippets into a growing loadedLectureText string, triggers the exam phase, and aggregates results into per-tier statistics displayed in the Unity Inspector and console. The exam is run using Unity coroutines with a semaphore (MAX_CONCURRENT_MCQ) to throttle concurrent API calls and avoid GPU out-of-memory on the Python side.

4.3.2 StudentAgent.cs - Per-Student HTTP Interface

Each StudentAgent encapsulates three API calls: SendLecture (POST /send_lecture) sends the transcript and receives back attention statistics; SubmitMCQ (POST /answer_mcq) sends a question and options and receives the chosen index, confidence, and no_context flag. Attention statistics are synced from the authoritative Python backend after SendLecture, overriding any local defaults.

4.3.3 BaseStudent.py - Attention Model and Context Management

BaseStudent implements the two-phase attention pipeline. The _apply_blanking() method first drops whole sentences with probability blank_prob, replacing them with [BLANKED] tokens. Then _apply_markov_mask() runs the two-state Markov

chain word by word across the remaining text, replacing zone-out bursts with a single [ZONED] token. The `get_clean_context()` method strips all tokens to produce the clean text that exclusively feeds the LLM. Statistics including `zone_out_episodes`, `avg_zoneout_length`, `theoretical_retention`, and `effective_retention` are tracked and returned to Unity after each `SendLecture` call.

4.3.4 Agent.py - FastAPI Student AI Server

`Agent.py` (port 5002) manages all student AI state via a dictionary of `BaseStudent` instances keyed by `student_id`. `/send_lecture` applies the attention pipeline and returns statistics. `/answer_mcq` retrieves the top-k context chunks via FAISS, re-ranks with the cross-encoder, and prompts Qwen2.5-0.5B to select an MCQ option; if no relevant context exists, a random option is returned with `no_context=True`. `/generate_lecture_question` takes a lecture snippet via a typed Pydantic model and uses the LLM to generate a natural student question. A separate `/testbank_similarity` endpoint computes cosine similarity between the current snippet and every test bank question using the embedding model, returning only questions above a configurable threshold - enabling semantic question selection rather than simple keyword matching.

4.3.5 VoiceController.cs and Agent_Voice.py - ASR Pipeline

`VoiceController` records the microphone continuously using Unity's `Microphone.Start()` API. Every 30 seconds it extracts the latest audio window, normalises it, converts it to WAV, and POSTs it to `Agent_Voice.py` (`/transcribe`). `Agent_Voice.py` passes the audio to Whisper, applies a filler-word cleanup regex,

and returns the cleaned transcription. When the lecture ends, the full recording is uploaded for a final pass.

4.3.6 QuestioningManager.cs and Agent_TTS.py - TTS Pipeline

QuestioningManager fires on a configurable cooldown (default: 60 s, after a 120 s warmup). In LLM mode it POSTs the current snippet to /generate_lecture_question and speaks the result. In Test Bank mode it selects a question by keyword overlap with the snippet. The text is sent to Agent_TTS (/synthesize); the returned WAV is decoded in Unity by a custom WavToAudioClip() parser and played through an AudioSource component.

4.4 Key Implementation Decisions

- Markov vs. random masking: Markov produces coherent bursts, while random masking was rejected because it does not reflect real zone-out patterns and makes repetition artificially less helpful.
- Sentence blanking before word masking: Sentence dropout captures coarser distraction (student checks phone, looks away); Markov captures finer within-sentence drift. Running them in sequence gives realistic mixed behaviour.
- Coroutine-based parallelism with semaphore: All students are tested concurrently in Unity coroutines, but the semaphore prevents more than MAX_CONCURRENT_MCQ simultaneous HTTP calls to avoid GPU VRAM memory exhaustion on the Python side.

4.5 Key Algorithms and Code Walkthrough

4.5.1 Two-Phase Attention Pipeline

The two-phase pipeline in BaseStudent.py is the computational core of the system. Phase one (_apply_blanking) splits the lecture into sentences using a regex on sentence-ending punctuation, then independently draws a Bernoulli sample for

each sentence against `blank_prob`. Sentences that fail the draw are replaced with the token `[BLANKED]`. This phase models coarse distraction - a student checking their phone, looking away - and is intentionally sentence-granular so that complete thoughts are removed rather than mid-sentence fragments, which would produce grammatically invalid context.

Phase two (`_apply_markov_mask`) runs a left-to-right pass over the token list. While `ATTENTIVE`, words are kept; each word has a `p_dropout` chance of triggering a `ZONED_OUT` transition. While `ZONED_OUT`, words are discarded and a single `[ZONED]` marker is emitted per burst; each word has a `p_recover` chance of returning to `ATTENTIVE`. After both phases, `get_clean_context()` strips all tokens, and the clean string is the sole input to the FAISS index and LLM prompt.

4.5.2 RAG Pipeline in `Agent.py (/answer_mcq)`

The `/answer_mcq` endpoint: retrieves the student's `BaseStudent` instance; chunks the clean context (150 words, 30-word overlap); embeds chunks with Sentence-BERT and builds a FAISS index; retrieves the top-5 chunks by embedding similarity; re-ranks with the cross-encoder and takes the top-3 as the final context window; constructs a prompt and sends to Qwen2.5-0.5B with an instruction to answer based only on the provided context. The generated token is parsed for a letter (A/B/C/D) and mapped to `choice_index`. If no relevant chunks are found, `no_context=True` and a random index is returned.

4.5.3 WAV Encode/Decode Pipeline

Unity has no built-in WAV encoder or decoder, requiring custom implementations. The encoder (`ConvertToWavFromSamples`) writes a standard PCM WAV header

followed by 16-bit PCM samples converted from Unity's float32 range, with a normalisation pass beforehand to prevent near-silent inputs.

The decoder (WavToAudioClip) reads the WAV header at fixed byte offsets, converts 16-bit integers back to float32, and loads them into an AudioClip via AudioClip.Create() and SetData() for immediate playback without requiring a disk or I/O.

5. Testing and Evaluation

5.1 Test Plan and Methodology

Testing proceeded in four phases: (1) unit testing of the Markov model in isolation, (2) integration testing of each API endpoint with synthetic inputs, (3) system-level experiments using fixed lecture inputs to measure tier separation and repetition effects, and (4) in-engine simulation testing inside the Unity VR environment with live microphone input.

5.2 Unit Testing - Markov Model

The Markov model was validated analytically. For each attention profile, 100 runs of the masking pipeline were applied to a 2000-word synthetic lecture, and the measured `effective_retention` was compared to the theoretical steady-state $P(\text{attentive})$. Results were within $\pm 5\%$ of theory in all cases, confirming correct implementation.

5.3 Validation of Context Isolation (NFR1)

A 25-question MCQ exam drawn entirely from a nutrition curriculum was administered to all agents after sending a climate change lecture. Every agent across all tiers scored exactly 0/25. This is the most important validation result: it proves the agents have zero background knowledge and answer strictly from context. The `no_context` flag was set for all agents, confirming that FAISS retrieval found no relevant chunks.

5.4 System-Level Experiment Results

Six controlled test variants were run using fixed lecture inputs (not live microphone) to isolate the attention model's behaviour:

Test Variant	Tier A (A/Perfect)	Tier B (Medium)	Tier C (Low)	Notes
Standard (Baseline)	21.7 / 25	17.0 / 25	10.7 / 25	Clear tier separation; all within expected ranges
Repeated 2×	23.7 / 25	19.0 / 25	13.3 / 25	All tiers improve; Tier C +2.6 pts
Repeated 5×	23.0 / 25	20.7 / 25	16.7 / 25	Tier C exceeds Tier B baseline
Truncated (end removed)	19.0 / 25	16.3 / 25	14.0 / 25	Q15–17, 19 all at 0%; removed section
Unrelated Content	0 / 25	0 / 25	0 / 25	Full guessing; context isolation confirmed
In-Engine (VR, live mic)	~23.0 / 25	~20.7 / 25	~16.7 / 25	End-to-end; latency within NFR3

5.5 Key Findings

- Tier Separation (NFR2 confirmed): In every test run, Tier A > Tier B > Tier C with no overlap in means.

- Repetition Sensitivity: Doubling the lecture content raised Tier C by 2.6 points; quintupling it raised Tier C above the original Tier B baseline (16.7 vs. 17.0). This matches Markov theory: lower-tier students benefit more from repetition because each repetition is an independent episode.
- Content Sensitivity: Removing the final section dropped the overall mean from 20.0 to 16.4, and exactly those questions covering the removed section scored 0%. The system can identify which lecture segments were not retained.
- Latency (NFR3 confirmed): A 25-question, 9-student exam completed in under 5 minutes on the test hardware.

6. Entrepreneurial / Innovation Aspects

6.1 Market Relevance

AI-powered analytics tools are among the fastest-growing segments. The VR Classroom Teacher Evaluation System addresses a specific and underserved niche: real-time, lecturer-facing feedback grounded in attention modelling rather than retrospective assessment.

Potential customer segments include: universities adopting AI-enhanced curriculum review, corporate training departments that require quantifiable lecture effectiveness metrics, and online education platforms seeking to simulate student engagement for course quality assurance.

6.2 Novelty and Innovation

The system's key innovations relative to the state of the art are:

- Burst-structured Markov attention model: Unlike uniform random masking, the Markov model produces psychologically realistic zone-out patterns with analytically predictable, non-overlapping tier ranges.
- Strict context isolation: All agents answer from personal filtered context only. No background knowledge contamination.
- Integrated live pipeline: ASR → attention → RAG → MCQ → TTS in a single VR scene, with no manual intervention required between lecture delivery and exam results.
- Repetition sensitivity as a feature: The system can quantify exactly how much a given concept needs to be repeated to reach a target retention rate in a given tier.

6.3 Commercialisation and Scalability

The Python backend is fully containerisable (Docker + GPU server). The Unity frontend can be extended to VR headsets (Meta Quest, HTC Vive) for full immersive deployment. A SaaS model is feasible: instructors upload lecture slides and a test bank, select a student count and attention distribution, and receive an instant analytics report. The local LLM can be replaced with a cloud API for deployments without GPU hardware.

6.4 Ethical and Societal Considerations

All student simulation is synthetic, as no real student data is collected or processed. The system does not profile or track individual real people. The attention parameters are based on published cognitive psychology research and are not intended to stereotype or label actual students. Care was taken to ensure that the system provides aggregate feedback to lecturers rather than individual

student surveillance. Future deployments should include transparency disclosures and opt-in consent if real microphone data from actual classrooms is ever used.

7. Project Management and Teamwork

7.1 Timeline and Milestones

Phase	Milestone
Semester 1	
Sep 2025	Project scoping and literature review, research gap identified, Markov model chosen over random masking
Oct 2025	Core attention model (BaseStudent.py), Markov chain implemented and validated analytically with tier ranges
Oct 2025	LLM + RAG pipeline, FAISS + cross-encoder integrated (Agent.py)
Oct 2025	End-to-end Colab simulation, full exam run with simulated students, context isolation confirmed, lecture repetition results validated
Semester 2	
January 2026	Unity frontend end-to-end HTTP flow between Unity and Python backend with first in-engine exam run
February 2026	Whisper ASR integration live lecture transcription pipeline with filler-word cleanup added
February 2026	Kokoro TTS students ask spoken questions, WAV playback in Unity confirmed though AudioSource
March 2026	Added classroom 3d environment, and replaced placeholder capsules with student Prefabs.
March 2026	Added PowerPoint Displayer To show lecture slides inside the virtual classroom.
April 2026	System debugging, fixing issues and testing
April 2026	Final report and submission

7.2 Resource Management

Development was carried out on a personal laptop (Intel i7-7700, 16 GB RAM, NVIDIA GTX 1050 4 GB). All AI models (Whisper base, Sentence-BERT, cross-encoder, Qwen2.5 0.5B, Kokoro) are open-source and run fully locally with no API costs. Unity 6 was used under its free personal license.

7.3 Individual Project - Roles

This is an individual capstone project. All design, implementation, testing, and documentation was performed by Ricardo Salloum.

8. Results and Discussion

8.1 Comparison with Objectives

All five project objectives were met:

- Markov attention model with tier separation: Implemented and analytically validated. Non-overlapping tier ranges confirmed across all six test variants.
- Full ASR → filtered context → RAG MCQ pipeline: Operational end-to-end, both in isolated tests and in-engine with live microphone.
- Strict context isolation: Confirmed by the unrelated content experiment (0/25 across all tiers).
- Repetition sensitivity: Confirmed; 5× repetition raises Tier C above Tier B baseline, consistent with Markov theory.
- Unity VR integration: Fully operational; exam runs in under 5 minutes for 9 students, 25 questions.

8.2 Strengths

- Analytically grounded: The Markov model has a closed-form steady-state solution, making its behaviour predictable and interpretable without empirical tuning.
- Strict isolation: The zero-score result on unrelated content is one of the strongest validation signals possible - it proves agents are not hallucinating or drawing on background knowledge.
- Modular architecture: Each Python server can be independently restarted, updated, or replaced without affecting the others.
- Real-time feedback: Exam statistics are available seconds after the lecture ends, enabling immediate lecturer reflection.

8.3 Limitations

- Markov parameters are hand-tuned: The p_{dropout} and p_{recover} values for each tier were set to match published cognitive psychology findings, but have not been calibrated against real student attention data from an actual classroom.
- LLM hallucination risk: While tested to be context-grounded in controlled experiments, the local LLM may occasionally produce plausible-sounding but incorrect answers on edge-case questions.
- Single language support: Whisper is configured for English only, the system cannot currently handle multilingual lectures.
- No real student validation: The system models attention mathematically; its predictions have not been compared against real student exam scores from a physical lecture.
- Domain scope, lecture-based subjects only: The system is well-suited to subjects where knowledge is primarily declarative and text-transmissible. For example it works well for K–12 general education, history, liberal arts, biology, nursing, and non-mathematical high school courses. It works because the correct answer to an MCQ question can, in principle, be

found verbatim or by close paraphrase in the lecture transcript. The system does not extend to problem-solving disciplines such as advanced mathematics, engineering, or computer science, where correct answers require multi-step symbolic computation rather than text retrieval. A student agent may correctly recall that a derivative represents a rate of change, but cannot compute the derivative of x^3 ; it may know that ice floats because its density is lower than water, but cannot derive that density from a given mass and volume. In these domains the RAG pipeline retrieves correct conceptual framing but the LLM cannot perform the procedural reasoning the question demands, producing near-random MCQ scores that do not reflect the attention model. This is an architectural constraint, not a tuning issue, and would require integration significantly larger reasoning model to address.

8.4 Contribution to Research and Practice

This project makes the following contributions: (1) a novel application of two-state Markov chains to word-level lecture attention simulation, with analytically derived tier separation; (2) empirical validation of context isolation in RAG-based educational QA; (3) a working demonstration that a full AI-in-the-loop educational feedback system can be built with open-source tools and consumer hardware; and (4) open-source code documenting all integration patterns (Unity-FastAPI HTTP, Whisper WAV upload, Kokoro WAV decode) that future developers can reuse.

9. Conclusion and Future Work

9.1 Summary of Contributions

This project designed, implemented, and validated a complete VR Classroom Teacher Evaluation System. The system's core innovation - a two-state Markov

attention model applied at the word level to real lecture transcripts - produces psychologically realistic, analytically predictable, and tier-separated exam score distributions. The integration of Whisper ASR, FAISS RAG, local LLM answering, and Kokoro TTS within a Unity VR scene demonstrates that a production-quality AI-in-the-loop educational simulation is achievable today with commodity hardware and open-source tools. The unrelated content experiment, in particular, provides strong empirical evidence that the system achieves genuine context grounding - a property that is both practically important and technically non-trivial to guarantee.

9.2 Future Work

- Real student calibration: Collect attention and exam data from real students in a controlled lecture, then fit Markov parameters to match observed score distributions. This would transform the system from a theoretically motivated simulation into an empirically validated model.
- Multi-language support: Enable Whisper's multilingual mode and add Arabic attention model support for LAU's bilingual teaching environment.
- Learning analytics dashboard: Build a web-based dashboard (Streamlit or React) that visualises tier scores, per-question difficulty, retention heatmaps over the lecture timeline, and repetition-gain curves.
- LLM upgrading: Replace the local quantized model with a larger open-weights model (LLaMA-3 70B) or cloud API for more nuanced MCQ reasoning, particularly on questions requiring inference rather than direct retrieval.

References

1. Bunce, D. M., Flens, E. A., & Neiles, K. Y. (2010). How long can students pay attention in class? A study of student attention decline using clickers. *Journal of Chemical Education*, 87(12), 1438–1443. <https://doi.org/10.1021/ed100409p>
2. Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>
3. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
4. Qwen Team, Alibaba Cloud. (2024). *Qwen2.5: A party of foundation models* (arXiv:2412.15115). arXiv. <https://arxiv.org/abs/2412.15115>
5. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. *Proceedings of the 40th International Conference on Machine Learning*, 28492–28518.
6. Risko, E. F., Anderson, N., Sarwal, A., Engelhardt, M., & Kingstone, A. (2012). Everyday attention: Variation in mind wandering and memory in a lecture. *Applied Cognitive Psychology*, 26(2), 234–242. <https://doi.org/10.1002/acp.1814>
7. ValemVR. (2023). *VR game jam template* [Unity project template]. GitHub. <https://github.com/ValemVR/VR-Game-Jam-Template>